

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ESCUELA DE CIENCIAS Y SISTEMAS

SISTEMAS OPERATIVOS 1 SECCIÓN N

ING. SERGIO MÉNDEZ

AUX. ALLEN ROMÁN

VACACIONES DEL SEGUNDO SEMESTRE 2024



PROYECTO 1

Plataforma de Monitoreo en GCP

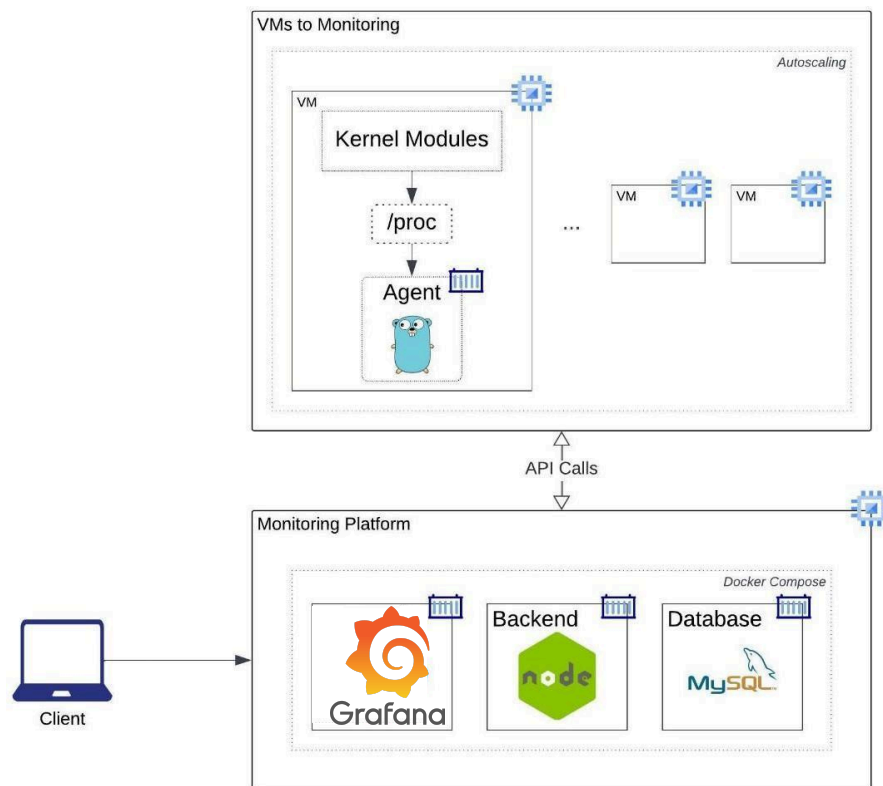
Objetivos

- Conocer el Kernel de Linux mediante módulos de C.
- Hacer uso de programación asíncrona con rutinas de Golang.
- Comprender el funcionamiento de los contenedores usando Docker.
- Utilizar GCP Compute Engine y autoscaling.

Introducción

En este proyecto, se tiene como objetivo principal implementar un sistema de monitoreo de recursos del sistema y gestión de procesos, empleando varias tecnologías y lenguajes de programación. El sistema resultante permitirá obtener información clave sobre el rendimiento del computador, procesos en ejecución y su administración a través de una interfaz amigable. Para el despliegue del proyecto se utilizará GCP Compute Engine y autoscaling para simular múltiples máquinas virtuales a monitorear.

Arquitectura



VMS a Monitorear

Kernel Modules

Módulo de RAM

Este módulo deberá de estar alojado en un archivo ubicado en el directorio /proc.

Características:

- Importar la librería <sys/sysinfo.h>, <linux/mm.h>
- La información que se mostrará en el módulo debe ser obtenida por medio de los struct de información del sistema operativo y no por otro medio.
- El nombre del módulo será: ram_<carnet>

Módulo de CPU

Este módulo deberá de estar alojado en un archivo ubicado en el directorio /proc.

Características:

- Importar la librería <linux/sched.h>, <linux/sched/signal.h>
- La información que se mostrará en el módulo debe ser obtenida por medio de los struct de información del sistema operativo y no por otro medio.
- El nombre del módulo será: cpu_<carnet>

Agente de Monitoreo

Este es un programa escrito en Golang y contenerizado, que es instalado en cada una de las VMs a monitorear. Este permite la comunicación entre la VM y la Plataforma de Monitoreo. Este cuenta con los siguientes componentes:

- **Recolector**

Se encarga de realizar llamadas a los módulos de Kernel por medio de rutinas para obtener la información del estatus de CPU y RAM. Así mismo este enviará la información al API de NodeJS de la Plataforma de Monitoreo para que sean almacenadas en la base de datos MySQL.

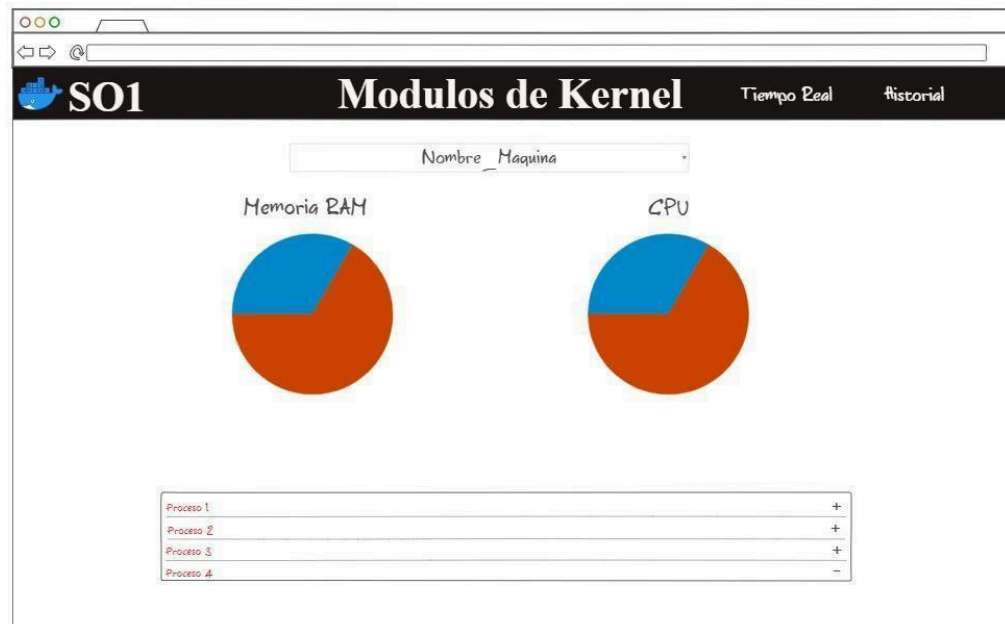
Plataforma de Monitoreo

Grafana

Monitoreo en Tiempo Real

En esta sección, se requiere mostrar datos en tiempo real, como el uso de CPU, RAM y una tabla de procesos. Grafana puede manejar esto mediante paneles personalizados y consultas dinámicas.

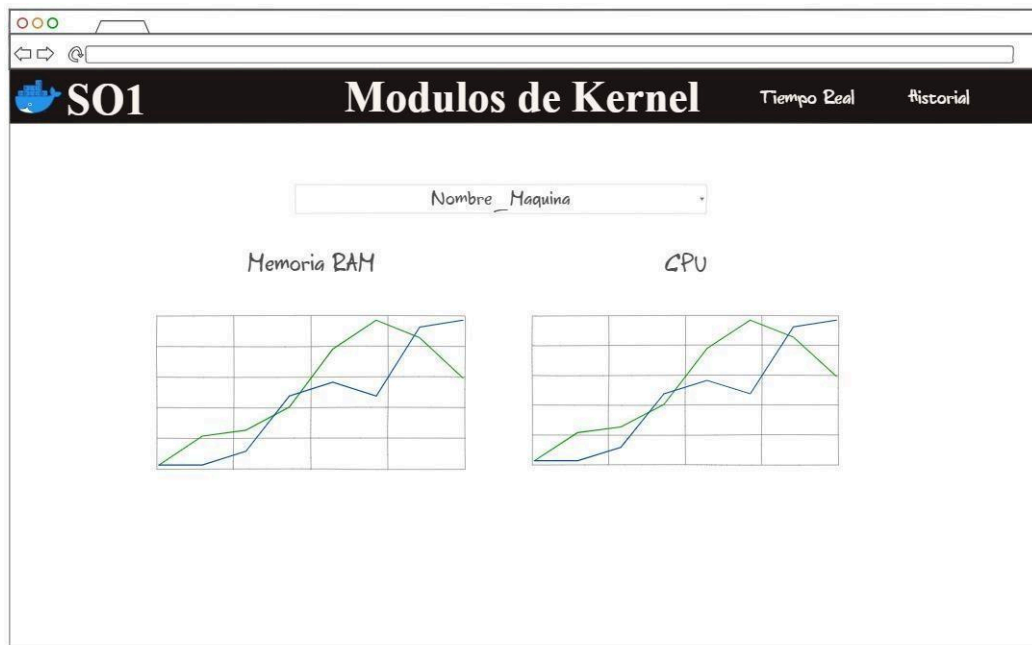
- Un Componente tipo Select en el cual se puede elegir la máquina de la cual se visualizarán los datos.
- Gráfica en Tiempo Real del porcentaje de utilización de la memoria RAM.
- Gráfica en Tiempo Real del porcentaje de utilización del CPU.
- Una tabla donde se muestran los Procesos, en esta se debe de mostrar la siguiente información para cada Proceso
 - PID, Identificador del Proceso
 - Nombre del Proceso
 - Usuario que ejecutó el Proceso
 - Estado en el que se encuentra el Proceso
 - %RAM que utiliza el proceso



Monitoreo a lo largo del Tiempo

En esta página se debe de Agregar:

- Un Select para elegir la máquina de la cual se visualizará la información.
- Una Gráfica del Rendimiento a lo largo del tiempo de la RAM.
- Una Gráfica de Rendimiento a lo largo del tiempo del CPU.



API NODEJS

Este servicio expone diversas APIs que posibilitan la comunicación con la base de datos. Dichas APIs serán empleadas tanto para la lectura de información como para la escritura de datos desde los agentes instalados en las máquinas sujetas a monitoreo.

Base de datos

Se debe implementar una base de datos MySQL por medio de un contenedor de Docker, esto para guardar los datos de los procesos, hijos, información de memoria ram y cpu. La base de datos debe de tener persistencia por lo que se requiere un Volumen de Docker para evitar que los datos se pierdan cada vez que el contenedor se reinicia.

Docker Compose

Para facilitar el despliegue de los contenedores de la Plataforma de Monitoreo se utilizará Docker Compose el cual permite gestionar múltiples contenedores en un solo bloque lo cual facilita la administración.

DockerHub

Se debe utilizar DockerHub para alojar las imágenes de los contenedores utilizados.

Google Cloud Platform

Plataforma de Monitoreo

Se deberá de crear una máquina virtual en GCP (Compute Engine) en la cual se levantará con ayuda de Docker-Compose Grafana, API de NodeJS y el contenedor de MYSQL.

VMs a monitorear

Se deberá de crear un Grupo de Instancias y se deberá de configurar el Autoscaling indicando la métrica de utilización a un 60%, como mínimo se tendrá 1 instancia y máximo 4. Y dentro de cada instancia debe de estar alojado los módulos de Kernel y el Agente para permitir la comunicación con la Plataforma de Monitoreo.

Restricciones

- El proyecto se realizará de forma individual.
- La obtención de la información se hará a través de los módulos de Kernel en C.
- El Agente de Monitoreo debe ser realizado con Golang.
- La API debe ser realizada con NodeJS.
- Grafana debe ser utilizado para el despliegue de datos.
- Las máquinas virtuales deberán ser de Ubuntu 20.04 o 22.04.
- Las imágenes de los contenedores deben de ser publicadas de DockerHub
- El proyecto debe de ser publicado en GCP usando Compute Engine y autoscaling

Github

- El código fuente debe de ser gestionado por un repositorio privado de Github
- Nombre del repositorio a utilizar en el curso: SO1_VAC_DIC_2024_<carnet>
- Crear una carpeta en el repositorio llamada Proyecto1.
- Agregar al auxiliar al repositorio de GitHub: Allenrovas

Calificación

- Al momento de la calificación se verificará la última versión publicada en el repositorio de GitHub
- Cualquier copia parcial o total tendrán nota de 0 puntos y serán reportadas al catedrático y a la Escuela de Ciencias y Sistemas.
- Si el proyecto se envía después de la fecha límite, se aplicará una penalización del 25% en la puntuación asignada cada 12 horas después de la fecha límite. Esto significa que, por cada período de 12 horas de retraso, se reducirá un 25% de los puntos totales posibles. La penalización continuará acumulándose hasta que el trabajo alcance un retraso de 48 horas (2 días). Después de este punto, si el trabajo se envía, la puntuación asignada será de 0 puntos.

Entregables

- La entrega se debe realizar antes de las 23:59 del 17 de diciembre de 2023.
- La forma de entrega es mediante UEDI subiendo el enlace del repositorio.
- Manual técnico con explicación de todos los componentes utilizados en el proyecto por ejemplo Grafana, Database, Módulos, GPC, etc. Este manual debe ser un archivo .md (Markdown), por lo que deberá estar en el README del repositorio.

Referencias

- [Docker](#)
- [Docker Compose](#)
- [GCP Compute Engine](#)
- [Autoscaling](#)